

INF-2204 Experiment project design

Distributions of Ranked Execution Time and Count across Programs and Platforms

Kathryn Ann Emblow
Department of Computer Science
UiT The Arctic University of Norway
 Tromsø, Norway
 kaemb0626@uit.no
 TheKeons@github

Adrian Eriksen
Department of Computer Science
UiT The Arctic University of Norway
 Tromsø, Norway
 aderi2118@uit.no
 kycksk@github

Tuva Marie Eaton Johansen
Department of Computer Science
UiT The Arctic University of Norway
 Tromsø, Norway
 tujoh3800@uit.no
 EarlGreyTT@github

I. INTRODUCTION

This report contains the project design of our experiment, where execution time and count of programs' code locations will be ranked to find the distributions that programs adhere to. Will different types of software and hardware show the same distribution? We hypothesise a common distribution family, with varying parameters. across all software, inputs, platforms, and code granularities.

A. Research questions

- 1) What distribution family and parameters result from ranking code locations by execution time and count, and do the rankings agree?
- 2) Is the distribution stable across hardware, OS, and run-time?
- 3) Is the distribution stable across different kinds of software and inputs?
- 4) Is the distribution stable across code granularity, such as instructions, lines, and functions?

II. HARDWARE ENVIRONMENT

Table I shows the hardware intended to be used in this project. It breaks down to three separate machines. A ThinkPad with Ubuntu, a ROG Zephyrus machine with dual boot Windows and Ubuntu, and a Raspberry Pi 5 with Debian's trixie. The goal was to have more than one machine to eliminate possible edge cases and to test the differences between instruction sets and microarchitectures.

The Raspberry Pi has ARM64, while the two laptops are x64, distinguished by their microarchitecture, where the ThinkPad has an Intel Core i5-10210U CPU, while the ROG Zephyrus has AMD Ryzen 7 6800HS. Differences include the cache policy, where the ThinkPad has an inclusive L3 while the ROG Zephyrus has a victim L3 policy [1] [3].

To ensure a stable hardware environment, the machine and OS is prepared by locking the CPU clock-frequency to the highest sustainable, shielding a core from interrupts and scheduling, turning off radios, hyper-threading for the core, swap and THP. This reduces measurement noise. The computers will also be connected to a power supply during the tests.

TABLE I
PRELIMINARY TABLE OF TEST PLATFORMS

Target	Platform		
	Hardware	OS	Runtime
ARM64 Linux	Raspberry Pi 5 Model B Rev 1.1	Raspberry Pi OS (Debian 13.6)	
x64 Linux	ThinkPad T14 Gen 1	Ubuntu 26.04.1 LTS	
x64 Linux	ROG Zephyrus G14 GA402RJ	Ubuntu 24.04.4 LTS	
JVM bytecode	ThinkPad T14 Gen 1	Ubuntu 26.04.1 LTS	JVM
JVM bytecode	ROG Zephyrus G14 GA402RJ	Ubuntu 24.04.4 LTS	JVM
JVM bytecode	Raspberry Pi 5 Model B Rev 1.1	Raspberry Pi OS (Debian 13.6)	JVM
JavaScript	ThinkPad T14 Gen 1	Ubuntu 26.04.1 LTS	V8 engine
JavaScript	Raspberry Pi 5 Model B Rev 1.1	Raspberry Pi OS (Debian 13.6)	V8 engine
JavaScript	ROG Zephyrus G14 GA402RJ	Ubuntu 24.04.4 LTS	V8 engine
Python	ThinkPad T14 Gen 1	Ubuntu 26.04.1 LTS	CPython
Python	Raspberry Pi 5 Model B Rev 1.1	Raspberry Pi OS (Debian 13.6)	CPython
Python	ROG Zephyrus G14 GA402RJ	Ubuntu 24.04.4 LTS	CPython

III. SOFTWARE ENVIRONMENT

Table I shows the Operating Systems intended to be used for this project. In practise there is little difference between them, since they all run the same kernel and distribution family with slightly different versions. Measurement preparation such as shielding a core from scheduling will further reduce differences. Some background jobs such as unattended updates are turned off, and the machines will not be running other software than what is strictly necessary for the testing.

Two tools are used for the actual measuring. Time is approximated by sampling with `perf` in multiple runs, while the execution count is measured using Valgrind's `callgrind` without tuning in a single run since it is deterministic.

IV. WORKLOADS

As the aim of the project is to compare distributions of execution time and count across programs and platforms, there will be a large number of different workloads that will be profiled. This means a diverse set of software of many different specializations. An incomplete list of possible workloads follows:

- Compression: XZ, zstd
- Browsers: Google Chrome
- Compilers: Clang / GCC
- Database management system: SQLite
- Visual/audio processing: ImageMagick / ffmpeg
- Git

V. DATASETS

The datasets will be chosen once the minutia in the Workload has been finalized. There will have to be enough data for their respective programs to run long enough for proper measurement.

VI. EVALUATION AND PERFORMANCE METRICS

To measure execution time and counts of programs, the code locations used are machine instruction instances. Exemplified in the code snippet below, there are three labels, L1 through L3. Each one counts as an individual code location, even though they have the same machine instruction, and L1 and L2 even share the same variables. However, their label demonstrates that the lines are at different locations, or offsets. They will be measured separately, both for time and count. Moreover, a loop over a location would accumulate the measurement of it.

```
L1: add x, y
L2: add x, y
L3: add y, z
```

As discussed in section III, `perf` will be used to sample the execution, in order to estimate a cumulative time for each code location. The exact number of times each one is run will be measured with Valgrind's tool `callgrind`. These will not be measured simultaneously, as the opposing measurements may be impacted by their measuring tools.

VII. EXPERIMENTAL PROTOCOLS

The project measures each program in one pass of runs, on each platform listed in Table I. The research questions are answered based on those data. Factors that can change between runs include platform, programs, their inputs, and code granularity. We fit several distribution families to the data to see which it belongs to and get the parameters. We check whether this stays consistent for count and time, and among different platforms, software and inputs, and code granularities. CPU clock-frequency is set to the highest sustainable per platform. Sampling rate is set to 100003 cycles; a prime to avoid measuring the same instruction in a loop. An arbitrary CPU core is chosen. Instruction attribution may be inaccurate since modern CPUs run out-of-order.

A `record` shell script is used to prepare the hardware and software environments, and measure a given program, as shown in figure 1. To prepare the environment, described in section II and III, `record` calls the shell script `tune`, which carries out the specifications of the tuning. Next, a `rank` script parses the outputs from `perf` and `callgrind`, and outputs a CSV file of time- or count-ranked code locations. The code locations run cumulatively the longest, or with the highest count, will respectively receive rank 1 for their metric. It should be noted that these shell scripts have been written with help from an LLM. As of yet, the number of times or length of each test has not been decided, but will be chosen once the workloads have been specified, in order to run a test until the aggregate of the runs converges.

A statistical analysis will be performed on the results, giving an average, variance and other metrics, however, this has not been written yet, but is included in the timeline.

VIII. TIMELINE

- | | |
|---------------|--|
| Week 40 & 41: | Collect software to test and write statistical analysis program |
| Week 41 - 44: | Testing stage |
| Week 44 & 45: | Data processing and analysis |
| Week 45 & 46: | Compare data to research question and hypothesis and write final paper |

IX. DISCUSSION

As discussed in the previous report [2] the original goal was to include Apple Silicon MacBooks in the research. After a lot of trial and error it was concluded that this is not possible. Apple are very restrictive on what information they publish for their machines. Meaning that there are no tools to get accurate data on instruction latency. Dougall Johnson has done research on Apple's microarchitecture [4]. This is based on black-box reverse engineering and is limited to the M1 CPU. The MacBook available to the researchers has a M3 CPU. It was discussed to only use the MacBook to count the instructions, however the only tool found that can count the instructions was `callgrind`. `Callgrind` is a Valgrind tool and does not have support for Apple Silicon MacBooks. This unfortunately removes a large part of the computer diversity, as a large part of the computer using population uses the newer

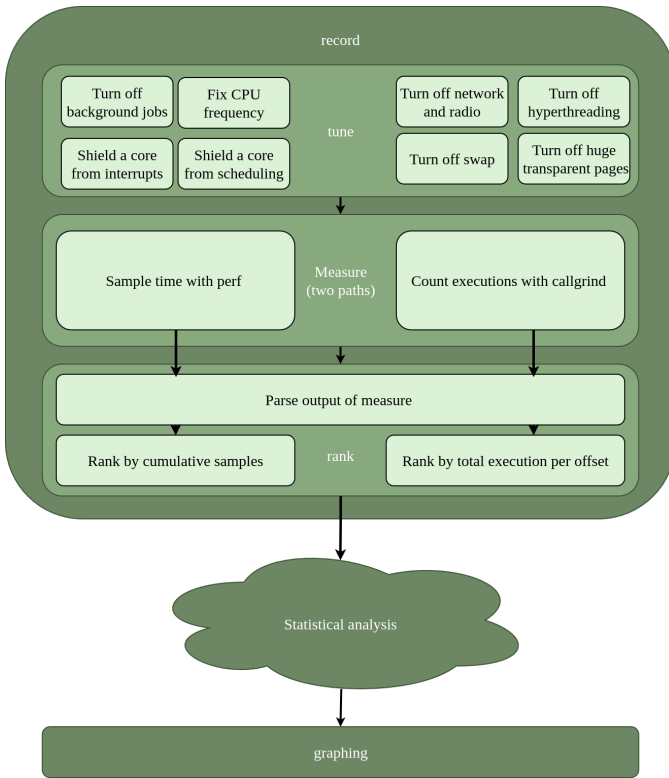


Fig. 1. Pipeline plan for data collection and analysis.

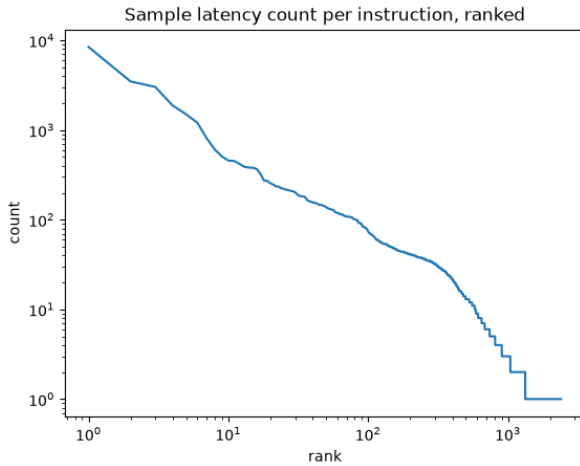


Fig. 2. Proof of concept graph, latency.

MacBooks. Luckily, it is still possible to test both latency and instruction count on the other ARM machines available to the researchers so this problem will be minimised to only MacOS. As it stands now, all Windows machines will also need to be excluded as perf is a Linux exclusive tool and Valgrind does not support Windows.

Figures 2 and 3 are proof of concept graphs. They were made by using real data from measuring both latency and count of the instructions from a quick-sort program. As these

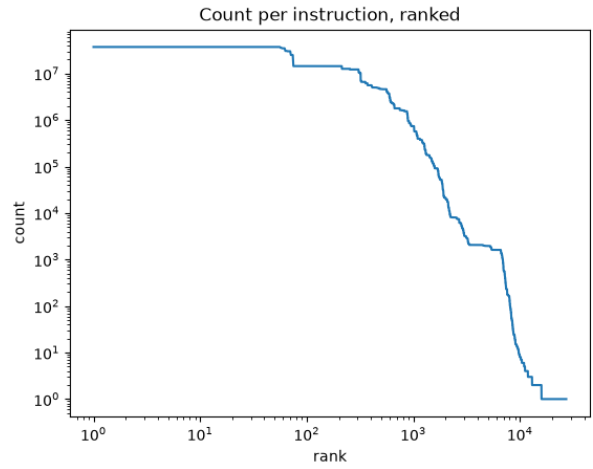


Fig. 3. Proof of concept graph, instruction count.

graphs were just to prove that the tools chosen work, they were not done in a noise free environment and will therefore not be analysed for anything other than bugs in the experimental protocol.

Research question four looks at more than just the machine instruction instances, as it also looks at other levels of code granularity. However, because of the profiling tools used, it is easy to fetch both the latency and count for lines and functions. This makes it so that research question four is implicitly taken care of in the experimental pipeline.

X. CONCLUSION

The results shown in figures 3 and 2 prove that the outlined project is possible to accomplish with the chosen tools. This along with the experiment plan detailed in section VII, point to a successful experiment portion of the planned paper.

REFERENCES

- [1] Advanced Micro Devices, Inc. *Software Optimization Guide for AMD EPYC™ 7003 Processors*. 56665. Version 3.00. Sec. 2.6. Nov. 30, 2020. URL: <https://docs.amd.com/v/u/en-US/56665> (visited on 09/28/2026).
- [2] Kathryn Ann Emblow, Adrian Eriksen, and Tuva Marie Eaton Johansen. *Ranked Distributions of Execution Time and Count across Programs and Platforms*. INF-2204 Experiment Project Proposal. Tromsø, Norway: Department of Computer Science, UiT The Arctic University of Norway, Sept. 2026. URL: <https://github.com/EarlGreyTT/INF-2204-Project/blob/main/reports/a1.pdf>.
- [3] Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual: Volume 1*. 248966-050US. Sec. 2.7.3, Table 2-16. Apr. 2024. URL: <https://cdrdv2.intel.com/v1/dl/getContent/671488> (visited on 09/28/2026).
- [4] Dougall Johnson. *Firestorm Overview*. <https://dougallj.github.io/applecpu/firestorm.html>. Apple Microarchitecture Research. Accessed: 2026-09-28. 2021.